KOGI

Independent Worker Operating System

Platform Architecture & Design Document

Version 1.0 • Confidential

Document Type	System Architecture Design Document
Platform	Kogi Independent Worker OS
Status	Draft — Internal Review
Scope	Portfolio, Workspace, WBS, Execution, Economics

1. Executive Summary

Kogi is an independent worker operating system — a unified, modular platform designed to give individuals, teams, and autonomous organizations the infrastructure they need to manage work, resources, capital, and collaboration at scale.

The platform integrates portfolio management, project execution, work breakdown structures, agile planning, communications, group economics, and AI-driven automation into a single cohesive system. Every user receives a root workspace, portfolio, and account provisioned automatically by the Kogi Kernel.

Mission	To provide every independent worker and autonomous organization with an operating system that orchestrates their work, capital, and community — from a single platform.

2. Platform Architecture Overview

The Kogi platform is organized into five horizontal layers, all orchestrated by a central Kernel. Each layer is modular, self-contained, and communicates through a unified event bus.

Layer	Responsibilities
Kernel	Central orchestrator — provisions modules, enforces security, manages I/O and inter-module communication
User Layer	Accounts, workspaces, and portfolios — the root objects provisioned to every user
Work Layer	Programs, projects, processes, executors, and outcomes — execution infrastructure
Planning Layer	WBS, stories, tasks, sprints, backlogs — strategic and agile planning
Resource Layer	Assets, capital, labor, investments, artifacts — resource management and economics
Tool Layer	Agile boards, scheduling, communications, idea studio — user-facing tools

2.1 High-Level Platform Map

```
KOGI PLATFORM
│
├── Kernel (Orchestrator + Provisioner)
│
├── User Layer
│   ├── Accounts
│   ├── Workspaces
│   └── Portfolios
│
├── Work Layer
│   ├── Programs  →  Projects  →  Processes  →  Outcomes
│
├── Planning Layer
│   ├── WBS  →  Work Packages  →  Themes  →  Initiatives
│   └── Epics  →  Stories  →  Tasks
│
├── Resource Layer
│   ├── Assets, Capital, Labor, Investments, Artifacts
│
└── Tool Layer
    ├── Agile Boards, Scheduling, Communications
    └── Idea + Concept + Prototyping Studio
```

3. Kernel

The Kernel is the central orchestrator and provisioner of the entire Kogi platform. It is responsible for managing all modules, enforcing security, scheduling processes, and routing communication between system components.

3.1 Kernel Responsibilities

Function	Description
Module Orchestration	Starts, stops, and coordinates all platform modules
Resource Provisioning	Allocates compute, memory, storage, and resources to users and modules
Process Scheduling	Schedules and dispatches processes across executors
Security Enforcement	Enforces access control, identity, and security policies platform-wide
Identity Management	Manages user identities, keys, tokens, and authentication
Storage Allocation	Manages data storage assignments and quotas
Inter-Module Communication	Routes messages and events between modules via event bus

3.2 Module Architecture

Every platform module is self-contained and exposes a standardized interface to the Kernel. Each module owns its own resources, memory, processes, storage, CLI commands, and security layer.

```
Module
│
├── Resources
├── Memory
├── Processes
├── Storage
│
├── CLI Commands
├── Configuration / Settings / Parameters
│
├── Services
├── Security
└── I/O System
```

4. User System

Every Kogi platform user receives three root objects upon registration, provisioned automatically by the Kernel: an Account, a Workspace, and a Portfolio.

4.1 Root Object Provisioning

```
New User Registration
        ↓
Account Created (identity, settings, keys, tokens)
        ↓
Workspace Created (hub, dashboard, tools, communications)
        ↓
Portfolio Created (items, resources, collections, directories)
        ↓
Resources Allocated (compute, memory, storage)
```

4.2 Account System

The account is the user's identity container. It holds all authentication, configuration, and access credentials.

Component	Description
User Profiles	Display name, avatar, biography, preferences
Settings & Options	Platform-wide configuration and UI preferences
Parameters	Behavioral and operational configuration values
Keys & Tokens	API keys, authentication tokens, session credentials
Sub / Linked Accounts	Linked personal or organizational account relationships

4.3 Workspace System

The workspace is the user's primary operating environment — the interface through which all Kogi tools, communications, and views are accessed.

Component	Description
Hub & Dashboard	Central access point and activity overview
User Tools	Agile boards, calendars, timelines, Gantt charts, roadmaps
Communications	Chat, messages, alerts, notifications

Component	Description
Idea & Prototype Studio	Idea capture, concept design, prototyping, and testing workspace
Views	Customizable views and dashboards for different perspectives

5. Portfolio System

The portfolio is the primary strategic container for a user or organization. It holds all items, resources, programs, projects, and collections. Portfolios are composable — they can contain sub-portfolios and be organized into collections and directories.

5.1 Core Object Model

```
Portfolio
│
├── Portfolio Items
│   ├── Programs
│   ├── Projects
│   ├── Sub-Portfolios
│   └── Resources
│
├── Collections  (grouped sets of items)
├── Directories  (ordered schedules of items)
└── Containers   (document/file organizers per item)
```

5.2 Portfolio Item Types

Item Types

Program	**Project**	**Sub-Portfolio**
Resource		

5.3 Resource Types

Resources represent assets and capabilities that can be attached to any portfolio item.

Resource Types

Artifacts	**Assets**	**Capital**
Land	**Estates**	**Labor**
Investments	**Accounts**	

5.4 Container Types

Portfolio items can be equipped with containers to organize their associated documents, notes, and files.

Container Types

Binder	**Book**	**Notepad**
Folder	**Briefcase**	

5.5 Portfolio Domains

Portfolios are domain-agnostic — they can represent any type of strategic asset or work context.

Domain	Use Case Examples
Solutions Portfolio	Systems, applications, platforms, services, products, goods
Social Network	Connections, community, friends, contact directory
Marketplace	Exchange items, inventory, listings, catalogues
Gigs & Contracts	Freelance tasks, contracts, deliverables
Investment Portfolio	Capital pools, ventures, equity stakes

6. Work Execution Model

Execution flows through a strict hierarchy: Programs contain Projects, Projects contain Processes. Processes are the atomic units of execution — they are running transformations operated by Executors that generate Outcomes.

6.1 Hierarchy

```
Portfolio
│
└── Programs
      │
      └── Projects
            │
            └── Processes
                  │
                  ├── Executions
                  ├── Actions
                  ├── Transformations
                  ├── Executors
                  └── Outcomes
```

6.2 Executor Types

Every process is carried out by one or more Executors. Executors can be autonomous AI agents, humans, hybrid combinations, or physical machines.

Executor Types

Agent	**Human**	**Hybrid**
Machine	**Custom**	

6.3 Outcome Types

Processes generate Outcomes — the measurable results of execution.

Outcome Types

Deliverable	**Result**	**Artifact**
Data	**State Change**	

7. Work Breakdown Structure (WBS)

The WBS system provides a structured decomposition of work from high-level strategic packages down to individual executable tasks. Every level of the hierarchy maps to a specific planning granularity.

7.1 WBS Hierarchy

```
WBS
│
└── Work Package
      │
      └── Theme
            │
            └── Initiative
                  │
                  └── Epic
                        │
                        └── Story
                              │
                              └── Task
```

7.2 WBS Level Definitions

Level	Definition
Work Package	Top-level strategic grouping of related themes and deliverables
Theme	A strategic area of focus grouping multiple initiatives
Initiative	A significant effort delivering multiple epics toward a goal
Epic	A large body of work decomposed into related stories
Story	A unit of user, business, or technical value (see Story System)
Task	An atomic, assignable unit of work with a clear completion criterion

8. Story System

Stories are the fundamental work and knowledge units of the Kogi platform. They represent anything from software features and bug reports to strategic goals, research spikes, and compliance audits. Stories link directly to tasks, executors, artifacts, and outcomes.

8.1 Story Object Model

```
Story
│
├── ID, Title, Description, Type, Status
├── Tasks
├── Executors
├── Artifacts
└── Outcomes
```

8.2 Story Types

The story type system covers the full spectrum of work, strategy, research, and operations.

Delivery

Feature	**Bug**	**Defect**
Capability	**Enabler**	**Blocker**

Quality & Compliance

Test	**Review**	**Audit**
Report	**Performance**	**Requirement**

Strategy & Operations

Strategy	**Tactic**	**Operation**
Business Case	**Mission**	**Vision**

Discovery & Design

Research	**Prototype**	**Spike**
Idea	**Concept**	**Design**

Planning & Goals

Milestone	Objective	Outcome
Risk	**Goal**	**Documentation**

9. Project Tracking System

The project tracking system provides a comprehensive set of views and tools for monitoring execution across agile, waterfall, and hybrid delivery models.

Tool	Purpose
Agile Boards	Kanban and scrum boards for sprint and flow-based delivery
Task Boards	Granular task management views
Sprints & Backlogs	Iteration planning, backlog grooming, and sprint management
Timelines & Gantt Charts	Visual scheduling and dependency management
Roadmaps	Strategic, quarter-level planning visualization
Schedules	Date-driven delivery schedules
Calendars	Time-based planning and event management

10. Group Economics & Crowdfunding Module

The Group Economics module provides Kogi users and organizations with infrastructure for collective capital formation, crowdfunding, cooperative investment, and shared ownership. Campaigns, capital pools, contributions, and ventures all appear as portfolio items.

10.1 Economic Models Supported

Economic Models

Crowdfunding	**Cooperative Investment**	**Community Treasury**
Mutual Aid Funds	**Collective Purchasing**	**DAO-Style Governance**
Revenue-Sharing	**Micro-Investment Groups**	

10.2 Core Objects

Object	Description
Capital Pool	Shared fund with contributors, governance rules, and pool type (community, investment, treasury, grant, mutual aid)
Campaign	A fundraising effort with a goal, deadline, and campaign type (donation, equity, revenue share, debt, grant)
Contribution	A funding event from a contributor: donation, investment, loan, membership, or stake
Ownership Stake	Tracks proportional ownership of a venture or asset post-funding
Venture / Asset	The funded entity: startup, real estate, community asset, product, platform, or infrastructure

10.3 Funding Lifecycle

```
Campaign Created
       ↓
Contributions Collected (held in escrow)
       ↓
Goal Reached?
   YES  →  Funds Released  →  Project Funded  →  Ownership Issued  →  Returns
Distributed
   NO   →  Refund to Contributors
```

10.4 Governance System

Each pool or campaign is governed by configurable rules. Supported voting systems include simple majority, stake-weighted, one-person-one-vote, quadratic, and delegated voting.

10.5 Security Layer

Security Controls

Escrow System	**Fraud Detection**	**Contribution Limits**
Identity Verification	**Multi-Sig Treasury**	**Governance Audit Logs**

11. Autonomous Organizations (AOs)

Autonomous Organizations (AOs) are self-governing entities on the Kogi platform. They combine portfolio management, governance, resource allocation, executor systems, and the event bus to operate independently with minimal human intervention.

11.1 AO Core Components

Component	Function
Governance Engine	Voting rules, approval workflows, AI-assisted decision making
Portfolio	Track ideas, projects, prototypes, and outcomes
Idea Management	Submission, versioning, prioritization, AI evaluation
Executor System	Human, AI agent, or hybrid execution of tasks
Resource Management	Allocate labor, tools, compute, and funding
AI Recommendation Engine	Predict risks, optimize assignments, suggest improvements
Event Bus	Real-time notifications and synchronization across contributors

11.2 Open-Source Community Use Cases

AOs are particularly well-suited for open-source communities, enabling autonomous capture of ideas, management of contributions, resource allocation, and iterative delivery.

Community Type	AO Workflow Highlights
Open Source Software	Idea capture → AI review → contributor execution → automated release → iteration
Open Hardware	Design proposals → resource allocation → prototype builds → testing → documentation
Data Science & AI	Experiment proposals → compute allocation → model training → validation → publishing
Creative & Media	Concept submission → AI-assisted asset generation → human review → release
Education	Lesson ideas → contributor authoring → AI accuracy checks → publication → iteration
Scientific Research	Experiment design → lab/compute allocation → execution → peer review → outcomes
Infrastructure	Improvement proposals → contributor assignment → AI anomaly detection → deployment

12. Event Bus

The Event Bus is the real-time communication backbone of the Kogi platform. All modules, AOs, and user actions emit and consume events through this unified system, enabling reactive workflows, notifications, and inter-module coordination.

12.1 Key Platform Events

Event	Description
user.provisioned	New user account, workspace, and portfolio created
portfolio.item_created	A new portfolio item has been added
process.started	A process execution has been initiated
process.completed	A process has finished and outcomes are available
story.status_changed	A story has moved to a new status
campaign.funded	A crowdfunding campaign has reached its goal
campaign.contribution_received	A new contribution has been made to a campaign
community.contributor_added	A new contributor has joined an AO community
community.release_published	A project or asset has been released publicly
community.iteration_completed	A delivery iteration has completed and roadmap updated

13. CLI Interface

The Kogi platform exposes a comprehensive command-line interface. Every module provides its own set of CLI commands, all routing through the Kernel's I/O system.

13.1 Portfolio Commands

```
kogi portfolio create          # Create a new portfolio
kogi portfolio add-item        # Add an item to a portfolio
kogi portfolio list            # List all portfolio items
kogi portfolio collection      # Manage collections
kogi portfolio directory       # Manage directories
```

13.2 Project & Work Commands

```
kogi project create            # Create a new project
kogi project track             # Open project tracking view
kogi wbs create                # Create a work breakdown structure
kogi story create              # Create a new story
kogi task assign               # Assign a task to an executor
```

13.3 Funding Commands

```
kogi fund create-campaign      # Launch a crowdfunding campaign
kogi fund contribute           # Contribute to a campaign
kogi fund create-pool          # Create a capital pool
kogi fund join-pool            # Join an existing pool
kogi fund distribute           # Distribute returns or revenue
kogi fund campaign-status      # Check campaign funding status
kogi fund treasury             # View treasury balances
```

14. Implementation Notes

The Kogi platform is designed for implementation in Zig, targeting a kernel-compatible, modular architecture. The following directory structure reflects the intended module organization.

14.1 Module Directory Structure

```
src/
├── kernel/
│   ├── kernel.zig
│   ├── scheduler.zig
│   ├── provisioner.zig
│   └── event_bus.zig
│
├── user/
│   ├── account.zig
│   ├── workspace.zig
│   └── provisioning.zig
│
├── portfolio/
│   ├── portfolio.zig
│   ├── item.zig
│   ├── resource.zig
│   ├── container.zig
│   └── collection.zig
│
├── work/
│   ├── program.zig
│   ├── project.zig
│   ├── process.zig
│   └── executor.zig
│
├── planning/
│   ├── wbs.zig
│   ├── story.zig
│   └── task.zig
│
└── crowdfunding/
    ├── campaign.zig
    ├── pool.zig
    ├── contribution.zig
    ├── stake.zig
    ├── venture.zig
    ├── governance.zig
    └── escrow.zig
```

14.2 Design Principles

- Modularity — every module is self-contained with its own resources, memory, and I/O
- Kernel-first — all provisioning, scheduling, and security flows through the Kernel
- Event-driven — loose coupling between modules via the Event Bus
- Composability — portfolios, AOs, and workflows are composable and extensible

- Executor-agnostic — processes can run on human, AI agent, machine, or hybrid executors
- Domain-agnostic — the portfolio and workspace models adapt to any industry or use case

Appendix: Glossary

Term	Definition
Portfolio	Primary strategic container for items, resources, programs, and collections
Portfolio Item	Fundamental object in a portfolio: program, project, sub-portfolio, or resource
Collection	A named grouping of portfolio items
Directory	An ordered schedule of portfolio items
Container	A document organizer attached to a portfolio item (binder, book, folder, etc.)
Program	A strategic grouping of related projects
Project	A time-bounded effort containing processes
Process	An atomic running execution, action, or transformation
Executor	The agent, human, machine, or hybrid that carries out a process
Outcome	The result generated by a completed process
WBS	Work Breakdown Structure — hierarchical decomposition from packages to tasks
Story	A unit of work or knowledge with a type, status, and linked tasks
AO	Autonomous Organization — a self-governing entity with portfolio, governance, and executors
Event Bus	Real-time message routing system connecting all platform modules
Kernel	Central platform orchestrator responsible for provisioning, scheduling, and security
Capital Pool	A shared fund with contributors, governance rules, and allocation logic
Campaign	A crowdfunding effort with a goal, deadline, and funding model
Venture	A funded entity: startup, asset, product, infrastructure, or creative project